

03 · N-gramas

Un **n-grama** es una secuencia de n palabras consecutivas. A diferencia de Bag of Words, los n-gramas **sí conservan el orden local**:

- **unigrama** ($n=1$): palabras sueltas
- **bigrama** ($n=2$): pares consecutivos
- **trigrama** ($n=3$): tríos consecutivos

Corpus: las **reseñas de entrega**, en `datos/resenas_entrega.csv`.

```
from pathlib import Path

import pandas as pd

# El texto ya minado de la tienda-virtual está copiado en la carpeta datos/ de
# este mismo proyecto:
# datos/resenas_entrega.csv   reseñas de entrega (post_compra)
# datos/comentarios.csv      testimonios / comentarios de clientes
# datos/productos.csv        catálogo con la descripción de cada producto
DATOS = Path("../datos")

def cargar(nombre, **kwargs):
    """Lee un CSV de la carpeta datos/ y lo devuelve como DataFrame."""
    ruta = DATOS / nombre
    if not ruta.exists():
        raise FileNotFoundError(f"No se encontró {ruta.resolve()}")
    print(f"Leyendo {ruta} ({ruta.stat().st_size / 1024:.1f} KB)")
    return pd.read_csv(ruta, **kwargs)
```

```
import re

resenas = cargar("resenas_entrega.csv")
corpus = resenas["texto"].dropna().astype(str).tolist()

def a_tokens(texto):
    # normalización ligera: minúsculas + solo palabras. Conservamos stopwords
    # porque para n-gramas el contexto ("llegó a tiempo") es justamente lo valioso.
    return re.findall(r"[a-záéíóúñü]+", texto.lower())

print(f"{len(corpus)} reseñas")
print("Ejemplo de tokens:", a_tokens(corpus[0])[:15])
```

```
Leyendo ../datos/resenas_entrega.csv (15.0 KB)
110 reseñas
Ejemplo de tokens: ['recibi', 'el', 'pedido', 'incompleto', 'faltaba', 'el', 'cable',
'de', 'carga', 'original', 'del', 'oneplus', 'dentro', 'de', 'la']
```

Extraer n-gramas con NLTK

`nltk.ngrams` genera las secuencias. Lo vemos sobre una frase real del corpus.

```
from nltk import ngrams

frase = a_tokens(corpus[2])
print("frase:", " ".join(frase), "\n")
for n in (1, 2, 3):
    grams = list(ngrams(frase, n))
    print(f"n={n} -> {len(grams)} {[ 'unigramas', 'bigramas', 'trigramas' ][n-1]}")
    print("    ", grams[:6])
```

frase: la segunda switch lite que regale llega perfecta y justo a tiempo para el cumpleaños que la necesitaba toda la experiencia de compra fue muy fluida

n=1 -> 26 unigramas

[('la',), ('segunda',), ('switch',), ('lite',), ('que',), ('regale',),]

n=2 -> 25 bigramas

[('la', 'segunda'), ('segunda', 'switch'), ('switch', 'lite'), ('lite', 'que'), ('que', 'regale'), ('regale', 'llego')]

n=3 -> 24 trigramas

[('la', 'segunda', 'switch'), ('segunda', 'switch', 'lite'), ('switch', 'lite', 'que'), ('lite', 'que', 'regale'), ('que', 'regale', 'llego'), ('regale', 'llego', 'perfecta')]

N-gramas más frecuentes en el corpus de reseñas

Con *stopwords* dentro, los bigramas más comunes son de relleno (`de la` , `con el`). Filtrando los n-gramas que contienen alguna *stopword* aparece el vocabulario **informativo** del corpus (`fecha estimada` , `día retraso` , `cable carga`).

```
from collections import Counter

import nltk

nltk.download("stopwords", quiet=True)
stop_es = set(nltk.corpus.stopwords.words("spanish"))

def top_ngramas(n, solo_contenido, k=12):
    cont = Counter()
    for texto in corpus:
        for g in ngrams(a_tokens(texto), n):
            if solo_contenido and any(w in stop_es for w in g):
                continue
            cont[g] += 1
    return cont.most_common(k)

print("Bigramas crudos (con stopwords):")
for g, c in top_ngramas(2, solo_contenido=False):
    print(f"  {' '.join(g):28} {c}")

print("\nBigramas de contenido (sin stopwords):")
for g, c in top_ngramas(2, solo_contenido=True):
    print(f"  {' '.join(g):28} {c}")

print("\nTrigramas de contenido (sin stopwords):")
```

```
for g, c in top_ngramas(3, solo_contenido=True):
    print(f" {' '.join(g):34} {c}")
```

Bigramas crudos (con stopwords):

de la	20
llego con	19
la caja	16
con la	14
tuve que	13
con el	10
en la	10
sin ningun	9
a tiempo	8
despues de	7
el equipo	7
con un	7

Bigramas de contenido (sin stopwords):

llego completo	7
llego bien	6
fecha estimada	6
ningun problema	4
galaxy s	4
robot aspiradora	4
bien empacado	3
cable anker	3
llego completa	3
compre llego	3
canon rebel	3
s ultra	3

Trigramas de contenido (sin stopwords):

galaxy s ultra	3
disco wd blue	3
cargador belkin llego	2
television tcl llego	2
camara instantanea llego	2
parlante jbl llego	2
llego bien empacado	2
ssd kingston llego	2
robot aspiradora llego	2
samsung the frame	2
ninguna notificacion previa	2
llego bien protegido	2

N-gramas con scikit-learn

`CountVectorizer(ngram_range=(2, 2))` construye directamente una matriz de bigramas; con `stop_words` descarta los de relleno. Sumando por columna se obtiene la frecuencia total de cada bigrama.

```
from sklearn.feature_extraction.text import CountVectorizer

vec = CountVectorizer(ngram_range=(2, 2), stop_words=list(stop_es))
X = vec.fit_transform(corpus)
frecuencias = X.sum(axis=0).A1
top = sorted(zip(vec.get_feature_names_out(), frecuencias), key=lambda x: -x[1])[:12]
```

```
pd.DataFrame(top, columns=["bigrama", "frecuencia"])
```

	bigrama	frecuencia
0	llego bien	7
1	llego completo	7
2	fecha estimada	6
3	dentro caja	5
4	llego dia	5
5	llego tiempo	5
6	dia retraso	4
7	galaxy s24	4
8	meses uso	4
9	ningun problema	4
10	robot aspiradora	4
11	acer aspire	3

Estimación de probabilidad (modelo de bigramas)

Un modelo de bigramas aproxima la probabilidad de una palabra a partir de la anterior:

$$P(w_i | w_{i-1}) \approx \frac{\text{count}(w_{i-1}, w_i)}{\text{count}(w_{i-1})}$$

Es decir: dado que apareció una palabra, ¿cuál es la más probable a continuación?

```
from collections import defaultdict

siguientes = defaultdict(Counter)
for texto in corpus:
    toks = a_tokens(texto)
    for a, b in ngrams(toks, 2):
        siguientes[a][b] += 1

def continuaciones(palabra, k=6):
    total = sum(siguientes[palabra].values())
    filas = [(w, n, n / total) for w, n in siguientes[palabra].most_common(k)]
    return pd.DataFrame(filas, columns=["siguiente", "count", "P(siguiete|'%s')"%
palabra])

continuaciones("llego") # el texto minado viene sin tildes
```

	siguiente	count	P(siguiete 'llego')
0	con	19	0.296875
1	completo	7	0.109375
2	bien	6	0.093750
3	sin	5	0.078125
4	a	5	0.078125
5	en	5	0.078125

Modelo generativo de n-gramas

Con las mismas probabilidades podemos *generar* texto: elegimos la siguiente palabra al azar según lo observado tras el prefijo de n-1 palabras. Es una versión minúscula de lo que, llevado al extremo con atención y Transformers, hacen los LLM actuales.

```
import random

class ModeloNgram:
    def __init__(self, n=3):
        self.n = n
        self.tabla = defaultdict(list)
        self.tokens = []

    def entrenar(self, textos):
        for texto in textos:
            toks = a_tokens(texto)
            self.tokens.extend(toks)
            for i in range(len(toks) - self.n + 1):
                prefijo = tuple(toks[i : i + self.n - 1])
                self.tabla[prefijo].append(toks[i + self.n - 1])
        return self

    def generar(self, inicio, largo=25):
        prefijo = tuple([inicio] * (self.n - 1))
        salida = [inicio]
        for _ in range(largo - 1):
            opciones = self.tabla.get(prefijo) or self.tokens
            salida.append(random.choice(opciones))
            prefijo = tuple(salida[-(self.n - 1):])
        return " ".join(salida)

random.seed(7)
modelo = ModeloNgram(n=3).entrenar(corpus)
for palabra in ("el", "llego", "producto"):
    print(f"[{palabra}] ->", modelo.generar(palabra, largo=25), "\n")
```

[el] -> el vuelto a diaria un de me sin un mis aspiradora sin experiencia día que sin leve completo retraso por un problema logístico que la

[llego] -> llego durante llego para un producto nuevo estoy esperando la respuesta del reclamo por garantía como parlante escribi con la caja sin defectos original este

[producto] -> producto veia muy fuera fue la mejor compra de tecnología que hice este año de la caja visiblemente golpeada en una esquina de la pantalla

El texto generado es localmente coherente (pares y tríos de palabras reales del corpus) pero se desvía sin rumbo: el modelo solo "recuerda" n-1 palabras. Word2Vec y los Transformers resuelven esa falta de contexto global.

Siguiente: 04_bow.ipynb .